Name of Student : Raghav Mahesh Sarda	**PRN:** 25070122***235***
Subject: Programming with Java	**Semester: III**
ASSIGNMENT No : 16	**Year: AY 2026-27**
TITLE : User-Defined Exceptions	

Problem Statement

Write a Java program to create and handle a user-defined exception.

Learning Objectives

To create and implement custom exceptions for application-specific error handling.

Program:

BankAccountSystem.java

```
// Assignment 16 - Exercise 1: Bank Account with User-Defined Exception
import java.util.*;
class InsufficientFundsException extends Exception // Custom
user-defined exception class
{
   InsufficientFundsException(String msg) // Constructor
   {
       super(msg); // Passing message to parent Exception class
   }
}

class BankAccount
{
   double balance;
   BankAccount(double bal) // Constructor
   {
       balance = bal; // Initializing balance
   }
   void withdraw(double amt) throws InsufficientFundsException // Method
throwing user-defined exception
   {
       if(amt > balance) // Insufficient balance condition
       {
```

```
            throw new InsufficientFundsException("Withdrawal failed:
Entered amount exceeds available balance!"); // Throwing custom
exception
        }
        else // Valid withdrawal
        {
            balance = balance - amt; // Deducting amount
            System.out.println("Withdrawal successful! Remaining Balance:
$" + balance); // Message
        }
    }
}

public class BankAccountSystem
{
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Initial Account Balance: ");
        double initialBal = sc.nextDouble();
        BankAccount acc = new BankAccount(initialBal); // Creating
account object

        System.out.print("Enter Amount to Withdraw: ");
        double withdrawAmt = sc.nextDouble();

        try
        {
            acc.withdraw(withdrawAmt); // Attempting withdrawal
        }
        catch(InsufficientFundsException e) // Catching user-defined
exception
        {
            System.out.println("Custom Exception Caught -> " +
e.getMessage()); // Handling custom exception
        }
        finally
        {
            System.out.println("Banking transaction completed."); //
Status message
            sc.close();
        }
    }
}
```

VotingRegistration.java

```
// Assignment 16 - Exercise 2: Age Validation with User-Defined
Exception
import java.util.*;
class InvalidAgeException extends Exception // User-defined exception
class
{
   InvalidAgeException(String msg) // Constructor
   {
       super(msg); // Passing error message to superclass
   }
}

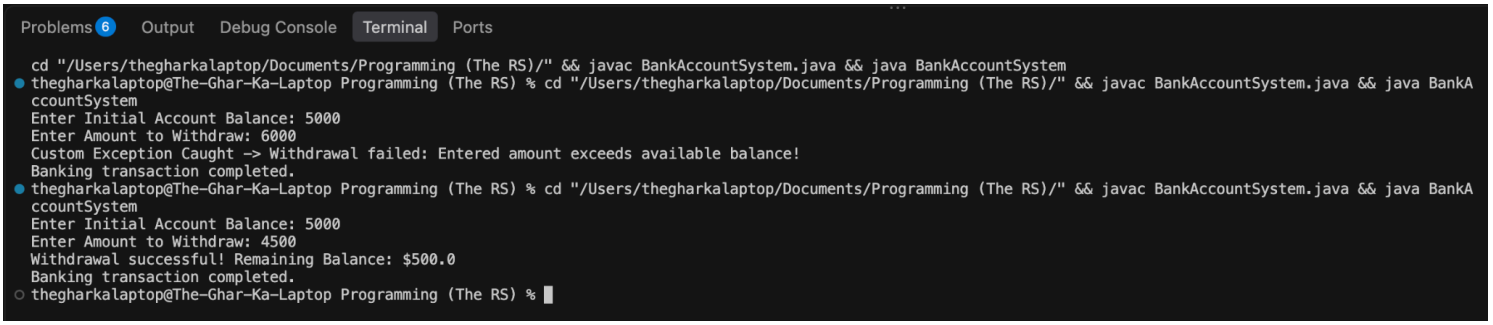
public class VotingRegistration
{
   static void checkEligibility(int age) throws InvalidAgeException //
Method declaring custom exception
   {
       if(age < 18) // Eligibility check condition
       {
           throw new InvalidAgeException("Candidate is under 18 years
old and ineligible to vote."); // Throwing custom exception
       }
       else // Valid age condition
       {
           System.out.println("Eligible");
       }
   }

   public static void main(String args[]) // Main Method
   {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter Candidate Age: ");
       int userAge = sc.nextInt();
       try
       {
           checkEligibility(userAge); // Calling validation method
       }
       catch(InvalidAgeException e) // Catching custom exception
       {
           System.out.println("Registration Rejected -> " +
e.getMessage()); // Error message
       }
       finally // Finally block
```

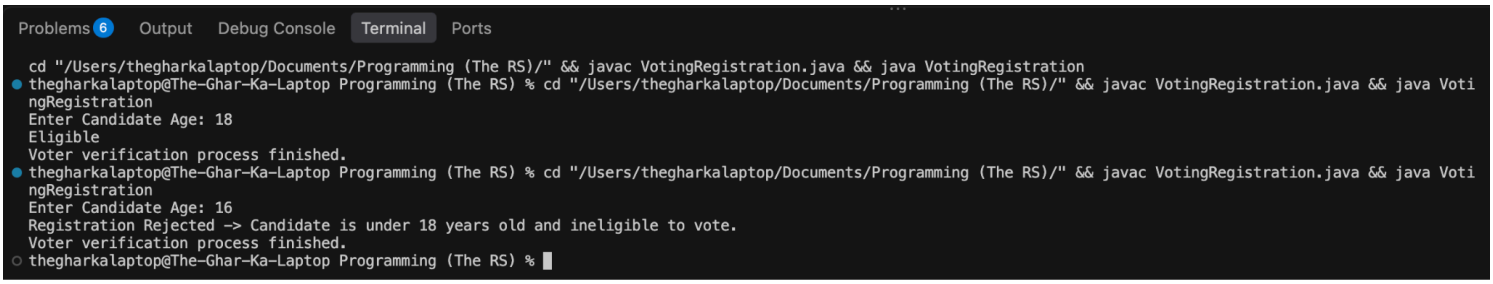
```
        {
            System.out.println("Voter verification process finished.");
            sc.close();
        }
    }
}
```

Output:

BankAccountSystem.java



VotingRegistration.java



Conclusion:

In this assignment, the concepts of custom error handling using user-defined exceptions were successfully implemented and verified. Custom exception classes extending the Exception base class provided domain-specific validation for banking transactions and age verification, enabling error reporting and maintaining program stability against improper user inputs.